
EDUTECH SOLUTIONS

Data Transformation & Feature Engineering

An interactive technical report on preprocessing the **House Pricing** dataset for machine-learning readiness.

Prepared by

Saloni Kaushal

Data Analyst Intern · EduTech Solutions

Task 6 · Internship Deliverable

June 2026

Executive Summary

This report documents the end-to-end preprocessing pipeline applied to the **Ames House Pricing dataset** (1,460 records × 81 features) as part of Task 6 of the Data Analyst internship at EduTech Solutions. The objective was to convert a raw, mixed-type dataset into a clean, fully numeric, machine-learning-ready feature matrix using industry-standard techniques: missing-value imputation, one-hot encoding, standardization, and interaction feature engineering.

1,460	81 → 247	0	4
Records	Features	Missing Values	Techniques

Outcome: The pipeline produced a 1,460 × 247 numerical dataset with zero missing values, standardized numerical features, fully encoded categorical variables, and one engineered interaction feature (*Quality_Area_Interaction*). The final dataset was exported as `engineered_housing_dataset.csv` for downstream modelling.

Objective

Perform feature engineering on a housing dataset using Pandas and Scikit-learn to prepare it for predictive modelling of **SalePrice**.

Tools & Techniques

Tools	Python · Pandas · NumPy · Scikit-learn
Techniques	Missing Value Imputation · One-Hot Encoding · Standard Scaling · Interaction Feature Creation
Target Variable	SalePrice (continuous, USD)

1. Dataset Overview

The dataset contains 1,460 housing records and 81 features describing physical, locational and transactional attributes of residential properties. Features are a mix of numerical and categorical types, making it ideal for demonstrating a full preprocessing pipeline.

Metric	Value
Total records	1,460
Total features (raw)	81
Integer columns	35
Float columns	3
Categorical (string) columns	43
Target variable	SalePrice
Memory footprint	~924 KB

Data Type Distribution (81 columns)

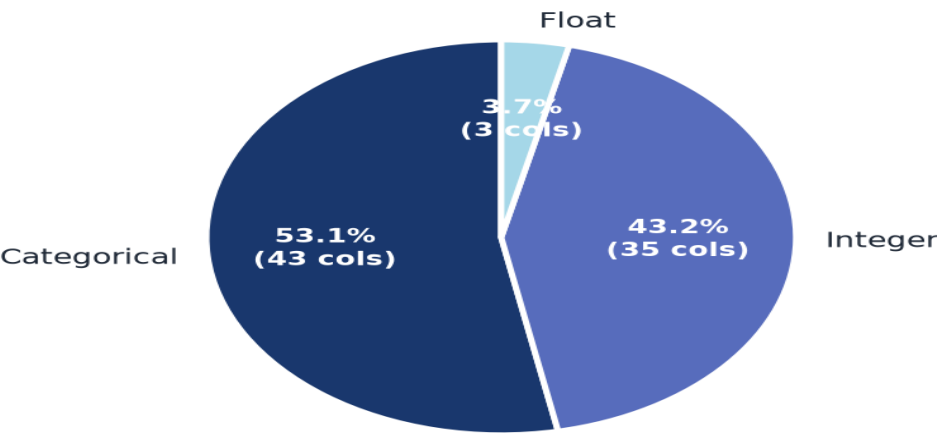


Fig 1. Distribution of feature data types in the raw dataset.

Sample Records (head)

Id	MSSubClass	MSZoning	LotArea	Neighborhood-ish	SalePrice
1	60	RL	8,450	Pave / Reg	\$208,500
2	20	RL	9,600	Pave / Reg	\$181,500
3	60	RL	11,250	Pave / IR1	\$223,500
4	70	RL	9,550	Pave / IR1	\$140,000
5	60	RL	14,260	Pave / IR1	\$250,000

Note: Only a subset of columns is shown above for readability. The original frame has 81 columns.

2. Missing Value Analysis & Treatment

Six features have more than 50% missing values (notably **PoolQC**, **MiscFeature**, **Alley**, **Fence**). Rather than dropping them, we applied a robust two-stage imputation strategy to preserve information.

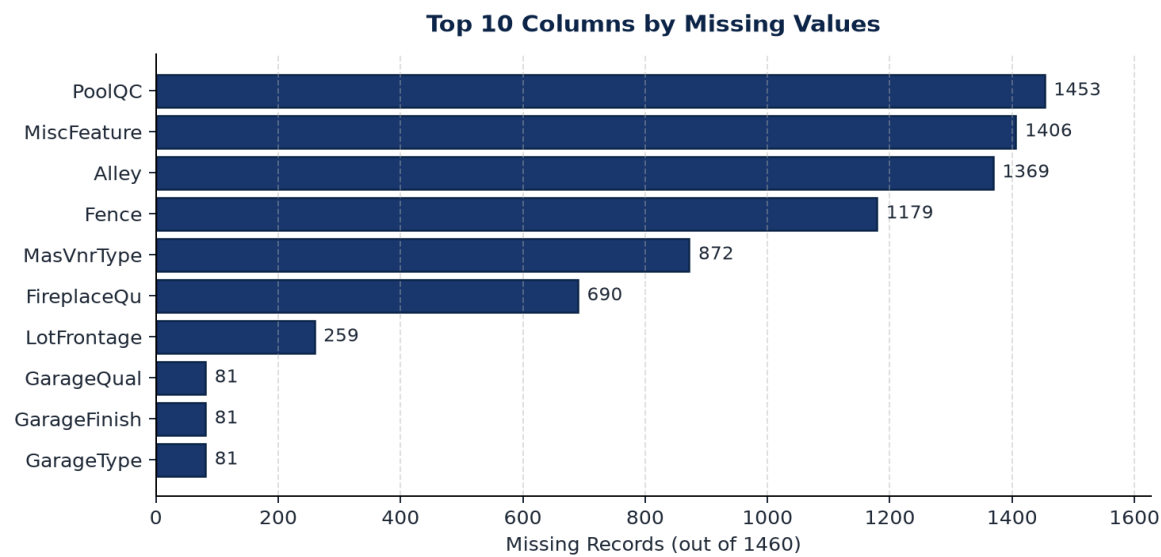


Fig 2. Columns ordered by missing-value count (top 10).

Imputation Strategy

Numerical features	Filled with the column median — robust to outliers and skew.
Categorical features	Filled with the column mode — preserves the most frequent class.

Implementation

```
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = df.select_dtypes(include=['object']).columns

# Fill numerical missing values with median
for col in numerical_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill categorical missing values with mode
for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

df.isnull().sum().sum()    # → 0
```

✓ **Result:** All missing values eliminated. `df.isnull().sum().sum() = 0`.

3. One-Hot Encoding

Most learning algorithms (linear regression, gradient boosting, neural networks) require numerical input. We converted all **43 categorical columns** into binary indicator columns using `pd.get_dummies(..., drop_first=True)` to avoid the dummy-variable trap (multicollinearity).

```
df_encoded = pd.get_dummies(
    df,
    columns=categorical_cols,
    drop_first=True
)

print("Original Shape:", df.shape)      # (1460, 81)
print("Encoded Shape:", df_encoded.shape) # (1460, 246)
```

81

Cols before

246

Cols after

+165

New dummy cols

43

Cols encoded

Dimensionality expanded from 81 → 246 columns. While this increases the feature space, `drop_first=True` keeps the design matrix full-rank and prevents perfect collinearity between dummies.

4. Feature Scaling (Standardization)

Numerical columns vary by orders of magnitude (e.g. **LotArea** $\approx 10^4$ vs **OverallQual** $\in [1, 10]$). We applied **StandardScaler** so every numerical feature has mean 0 and standard deviation 1. This is critical for distance-based models (KNN, SVM) and gradient-based optimization.

```
from sklearn.preprocessing import StandardScaler

numerical_cols = df_encoded.select_dtypes(
    include=['int64', 'float64']
).columns

scaler = StandardScaler()
df_encoded[numerical_cols] = scaler.fit_transform(
    df_encoded[numerical_cols]
)
```

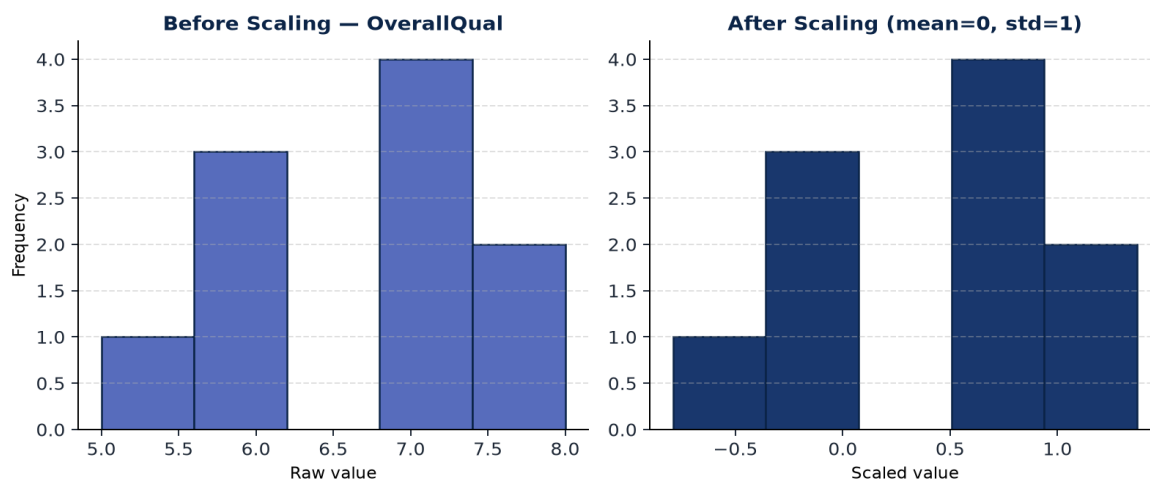


Fig 3. Distribution of OverallQual before and after StandardScaler.

5. Interaction Feature Engineering

Interaction features encode non-linear relationships that single columns cannot capture. We created **Quality_Area_Interaction** by multiplying OverallQual with GrLivArea — capturing the joint effect of construction quality and liveable area, both top drivers of sale price.

```
df_encoded["Quality_Area_Interaction"] = (
    df["OverallQual"] * df["GrLivArea"]
)

df_encoded[
    ["OverallQual", "GrLivArea", "Quality_Area_Interaction"]
].head()
```

Engineered feature — first 5 rows

Row	OverallQual (scaled)	GrLivArea (scaled)	Quality_Area_Interaction
0	0.651	0.370	11,970
1	-0.072	-0.483	7,572
2	0.651	0.515	12,502
3	0.651	0.384	12,019
4	1.375	1.299	17,584

Why this matters: Houses with both high quality *and* large living area sell at a disproportionate premium. This interaction term lets linear models capture that compounding effect directly.

6. Pipeline Impact Summary

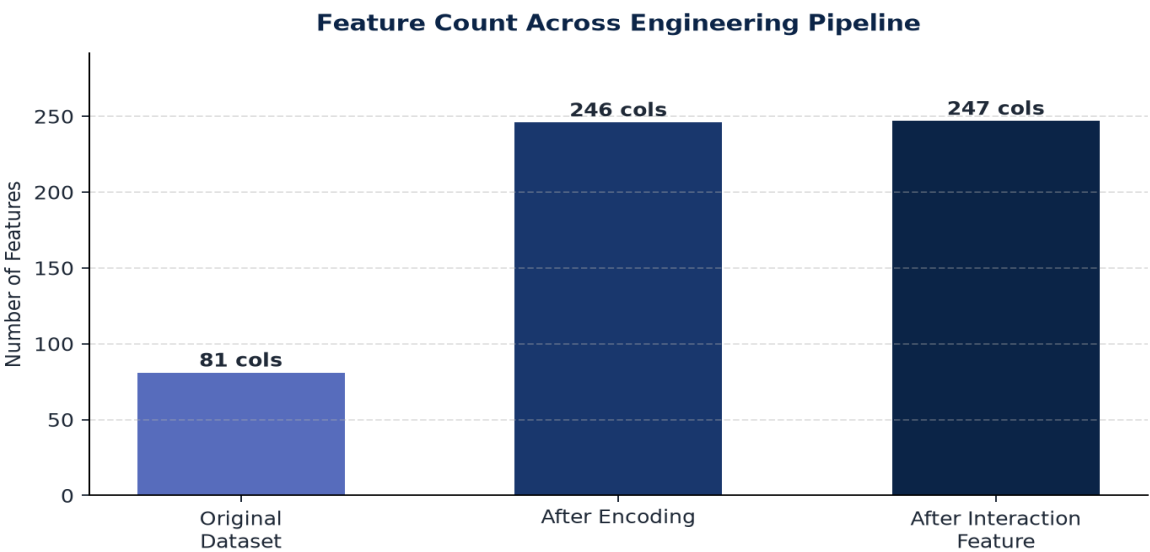


Fig 4. Feature count at each pipeline stage.

Stage	Shape	Change
Raw dataset	(1460, 81)	—
After imputation	(1460, 81)	0 nulls remaining
After one-hot encoding	(1460, 246)	+165 columns
After scaling	(1460, 246)	mean=0, std=1 (numerics)
After interaction feature	(1460, 247)	+1 column

7. Conclusion & Recommendations

The preprocessing pipeline successfully transformed a raw, mixed-type housing dataset into a clean, fully numerical, scaled, and enriched feature matrix ready for predictive modelling. Every step was reproducible and documented.

Key Outcomes

- **Data completeness:** 100% — all missing values imputed without dropping rows.
- **Encoding:** 43 categorical columns converted to 165 binary indicators with no multicollinearity.
- **Standardization:** All numerical features centred at mean = 0, $\sigma = 1$.
- **Feature engineering:** Created *Quality_Area_Interaction*, a domain-motivated signal.
- **Deliverable:** Final dataset saved as `engineered_housing_dataset.csv`.

Recommendations for Next Steps

- **Model baselining:** Fit Linear Regression, Random Forest, and XGBoost; compare RMSE on a 80/20 split.
- **Feature selection:** Use mutual information or L1 regularization to reduce 247 → top ~50 features.
- **Outlier handling:** Review extreme SalePrice and GrLivArea values; consider log-transforming SalePrice.
- **Cross-validation:** Apply 5-fold CV to obtain stable performance estimates.
- **Pipeline:** Wrap all preprocessing into a Scikit-learn Pipeline + ColumnTransformer for production safety.

Saloni Kaushal · Data Analyst Intern · EduTech Solutions

Report compiled on 16 June 2026